# P1239R0
# Placed Before

Published Proposal, 2018-10-07

**This version:**
http://wg21.link/P1239

**Author:**
Daniel Lustig (NVIDIA)

**Audience:**
SG1

**Project:**
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

**Source:**
https://github.com/daniellustig/P1239/blob/master/placed-before.bs

---

## Abstract

Adding "placed before" to C++

## Table of Contents

# § 1. Overview

There has been much interest in supporting lightweight, fine-grained memory ordering for syntactic dependencies in C++. Support for such ordering theoretically exists in the form of `memory_order_consume`, but as is now well known, no compiler to date has been able to support such behavior. This has been discussed in many previous papers, e.g., [p0098r1], [p0190r3], [p0462r1].

A related and notoriously difficult issue is the so-called "out-of-thin-air" (OOTA) problem: computations that somehow circularly depend on themselves can be made to return any arbitrary value, with the circular dependency causing speculation to become self-satisfying. This has also been discussed in many previous papers, e.g., [n3710], [ghosts]. The current C++ specification [n4750] places the burden of avoiding OOTA on the implementation:

> Implementations should ensure that no "out-of-thin-air" values are computed that circularly depend on their own computation.

In modern practice, it is generally considered impractical for implementations to enforce this guarantee, and so although research in the area continues, the current specification wording is considered insufficient.

Previous papers and discussions have suggested various solutions to the dependency ordering and OOTA problems. One approach, from [n3710] and elsewhere, is to globally enforce all atomic-load-to-

atomic-store ordering. A finer-grained variant of this approach is to introduce a `memory_order_load_store` that is intermediate in strength between `memory_order_relaxed` and `memory_order_acquire`. (Alternatively, `memory_order_relaxed` might be required to enforce load-to-store ordering, and a new `memory_order` would be introduced to provide the unsafe weaker behavior where appropriate.) However, this approach has two drawbacks: 1) particularly in the first variant, it will overconstrain the implementation in cases not prone to OOTA, and 2) it does not capture load-to-load dependency ordering semantics. A second class of solution involves building libraries that play clever tricks with inline assembly in order to propagate dependencies through the compiler; this was discussed in [p0750r1] and partially implemented in [WebKit], for example. This approach shows that libraries may be able to enforced fine-grained load-to-load and/or load-to-store dependency ordering, albeit at the cost of requiring a more complex API that `memory_order_consume` alone would require.

Our "placed-before" proposal attempts to integrate fine-grained dependency ordering enforcement mechanisms into the C++ memory model, as follows:

- **Fine-grained intra-thread memory ordering annotations**: We define a new order "placed before" that captures any and all intra-thread ordering semantics that the implementation is known to be able to maintain, including any or all of the proposals discussed above. Then, to first order, we perform a search and replace of "carries a dependency to" (which has proven impractical to use) with "placed before" (which is both more generic and more practical) in order to integrate "placed before" cleanly into the existing model.
- **A fix for OOTA**: We define "carries an unenforced dependency to" to include any "carries a dependency to" ordering that does not overlap "placed before". Then, we declare that loads caught in a cycle of "reads-from" and "carries an unenforced dependency to" (i.e., in an OOTA-prone execution) return unspecified values. This approach shifts the burden of avoiding OOTA from the implementation to the programmer, and better reflects the reality of today's implementations.

## § 2. Wording as of C++17

For reference, we quote the relevant wording as currently stated in [n4750].

Section 6.8.2.1 [intro.races] paragraph 7 defines "carries a dependency":

> An evaluation A carries a dependency to an evaluation B if
>
> - the value of A is used as an operand of B, unless:
>
>   - B is an invocation of any specialization of std::kill_dependency (32.4), or
>   - A is the left operand of a built-in logical AND (&&, see 8.5.14) or logical OR (||, see 8.5.15) operator, or
>   - A is the left operand of a conditional (?:, see 8.5.16) operator, or
>   - A is the left operand of the built-in comma (,) operator (8.5.19); or
> - A writes a scalar object or bit-field M, B reads the value written by A from M, and A is sequenced before B, or
> - for some evaluation X, A carries a dependency to X, and X carries a dependency to B.
>
> [ Note: "Carries a dependency to" is a subset of "is sequenced before", and is similarly strictly intra-thread. —end note ]

The definition of "carries a dependency to" continues to evolve and to be debated. See, e.g., [p0190r3] for further discussion. To the best of our knowledge, our proposal is compatible with any and all such changes, since the implementation is *not* required to track it under our proposal.

Section 6.8.2.1 [intro.races] paragraphs 8-10 define how "carries a dependency to" feeds into "dependency-ordered before", "inter-thread happens before", and "happens before":

An evaluation A is dependency-ordered before an evaluation B if

- A performs a release operation on an atomic object M, and, in another thread, B performs a consume operation on M and reads a value written by any side effect in the release sequence headed by A, or
- for some evaluation X, A is dependency-ordered before X and X carries a dependency to B.

[ Note: The relation "is dependency-ordered before" is analogous to "synchronizes with", but uses release/consume in place of release/acquire. —end note ]

An evaluation A inter-thread happens before an evaluation B if

- A synchronizes with B, or
- A is dependency-ordered before B, or
- for some evaluation X
    - A synchronizes with X and X is sequenced before B, or
    - A is sequenced before X and X inter-thread happens before B, or
    - A inter-thread happens before X and X inter-thread happens before B.

[ Note: The "inter-thread happens before" relation describes arbitrary concatenations of "sequenced before", "synchronizes with" and "dependency-ordered before" relationships, with two exceptions. The first exception is that a concatenation is not permitted to end with "dependency-ordered before" followed by "sequenced before". The reason for this limitation is that a consume operation participating in a "dependency-ordered before" relationship provides ordering only with respect to operations to which this consume operation actually carries a dependency. The reason that this limitation applies only to the end of such a concatenation is that any subsequent release operation will provide the required ordering for a prior consume operation. The second exception is that a concatenation is not permitted to consist entirely of "sequenced before". The reasons for this limitation are (1) to permit "inter-thread happens before" to be transitively closed and (2) the "happens before" relation, defined below, provides for relationships consisting entirely of "sequenced before". —end note ]

An evaluation A happens before an evaluation B (or, equivalently, B happens after A) if:

- A is sequenced before B, or
- A inter-thread happens before B.

The implementation shall ensure that no program execution demonstrates a cycle in the "happens before" relation. [ Note: This cycle would otherwise be possible only through the use of consume

> operations. —end note ]

Section 32.4 [atomics.order] paragraph 1.3 describes the current status of `memory_order_consume`:

> memory_order::consume: a load operation performs a consume operation on the affected memory location. [ Note: Prefer memory_order::acquire, which provides stronger guarantees than memory_order::consume. Implementations have found it infeasible to provide performance better than that of memory_order::acquire. Specification revisions are under consideration. —end note ]

Section 32.4 [atomics.order] paragraphs 9-10 describe out-of-thin-air values:

> Implementations should ensure that no "out-of-thin-air" values are computed that circularly depend on their own computation. [ Note: For example, with x and y initially zero,
>
> ```
> // Thread 1:
> r1 = y.load(memory_order::relaxed);
> x.store(r1, memory_order::relaxed);
> // Thread 2:
> r2 = x.load(memory_order::relaxed);
> y.store(r2, memory_order::relaxed);
> ```
>
> should not produce r1 == r2 == 42, since the store of 42 to y is only possible if the store to x stores 42, which circularly depends on the store to y storing 42. Note that without this restriction, such an execution is possible. —end note ]
>
> [ Note: The recommendation similarly disallows r1 == r2 == 42 in the following example, with x and y again initially zero:
>
> ```
> // Thread 1:
> r1 = x.load(memory_order::relaxed);
> if (r1 == 42) y.store(42, memory_order::relaxed);
> // Thread 2:
> r2 = y.load(memory_order::relaxed);
> if (r2 == 42) x.store(42, memory_order::relaxed);
> ```
>
> —end note ]

## § 3. "Placed Before"

"Placed before" is a new relation that captures any user-enforced intra-thread ordering. We define "placed before" as follows:

> An evaluation A is placed before an evaluation B if A is sequenced before B and one or more of the following hold:
>
> - A is an acquire operation, and A is sequenced before B
> - A is a load, A is sequenced before an acquire fence F, and F is sequenced before B
> - A is sequenced before a release fence F, F is sequenced before B, and B is a store
> - A is sequenced before B, and B is a release operation
> - A is a consume operation, and A carries a dependency to B
>
> [ Note: "Placed before" is a subset of "sequenced before", and is similarly strictly intra-thread. — end note ]

The consume operation is included here for completeness. If `memory_order_consume` is deprecated, it can simply be removed. If `memory_order_consume` is promoted to `memory_order_acquire` following standard practice today, then it simply overlaps with the first bullet. If (magically) someone were to make `memory_order_consume` work as originally intended, then its inclusion into "placed before" would allow it to continue serving its exact original purpose. The rest of this proposal is agnostic to any of these options.

The "placed before" formulation admits any number of fine-grained, thread-local ordering mechanisms. Examples of future inclusions into "placed before" might include (possibly seven letter variants of):

- `memory_order_load_load`
- `memory_order_load_store`
- Ideas such as the `dependent<T>` class in [p0750r1]
- Read-copy-update libraries

All of these may be useful in certain cases as lighter-weight alternatives to `memory_order_acquire` where the ordering needs are known to be finer-grained. The `memory_order_load_store` annotation would serve as a safe OOTA-free default, as discussed in §5.3 The "OOTA-Free-for-Placed-Before" Theorem. Examples for the `memory_order_load_load` might include uses of `smp_rmb()` in Linux.

## § 4. Fine-Grained Thread-Local Ordering

Although implementations have been unable to track dependencies being carried through code, the "carries a dependency to" relation itself appears to remain a good fit within the broader memory model. As such, we propose to simply replace "carries a dependency to" with "is placed before", and then to adjust other rule definitions accordingly, as described below.

By analogy to "dependency-ordered before", we define a new relation "inter-thread-placed before" that pairs release operations with "placed before". To do this, we duplicate the definition of "dependency-ordered before" from Section 6.8.2.1 paragraph 9 and modify it as follows:

> An evaluation A is ~~dependency-ordered before~~ inter-thread-placed before an evaluation B if
>
> - A performs a release operation on an atomic object M, and, in another thread, B ~~performs a consume operation on M and~~ reads a value written by any side effect in the release sequence headed by A, or
> - for some evaluation X, A is ~~dependency-ordered before~~ inter-thread-placed before X and X ~~carries a dependency to~~ is placed before B.
>
> [ Note: The relation ~~"dependency-ordered before"~~ "inter-thread-placed before" is analogous to "synchronizes with", but uses ~~release/consume~~ release and placed before in place of release/acquire. —end note ]

(Side note: [p0735r0] proposes to replace ~~"a value written by any side effect in the release sequence headed"~~ with "the value written". To the best of our knowledge, our proposal is agnostic to this change.)

We then replace "dependency-ordered before" with "inter-thread-placed before" in Section 6.8.2.1 paragraph 9:

> An evaluation A inter-thread happens before an evaluation B if
>
> - A synchronizes with B, or
> - A is ~~dependency-ordered before~~ inter-thread-placed before B, or
> - for some evaluation X
>
>   - A synchronizes with X and X is sequenced before B, or
>   - A is sequenced before X and X inter-thread happens before B, or
>   - A inter-thread happens before X and X inter-thread happens before B.
>
> [ Note: The "inter-thread happens before" relation describes arbitrary concatenations of "sequenced before", "synchronizes with" and ~~"dependency-ordered before"~~ "inter-thread-placed before" relationships, with two exceptions. The first exception is that a concatenation is not permitted to end with ~~"dependency-ordered before"~~ "inter-thread-placed before" followed by "sequenced before". The reason for this limitation is that ~~a consume operation participating in a "dependency-ordered before" relationship provides ordering only with respect to operations to which this consume operation actually carries a dependency~~ an atomic operation participating in a "placed before" relationship provides ordering only with respect to those specific operations that it is "placed before". The reason that this limitation applies only to the end of such a concatenation is that any subsequent release operation will provide the required ordering for a prior ~~consume operation~~ atomic operation at the head of a "placed before" relation. The second exception is that a concatenation is not permitted to consist entirely of "sequenced before". The reasons for this limitation are (1) to permit "inter-thread happens before" to be transitively closed and (2) the "happens before" relation, defined below, provides for relationships consisting entirely of "sequenced before". —end note ]

Now, any form of "placed before" can be used where "carries a dependency to" would have been used before.

## § 5. Repairing Out-Of-Thin-Air

The "placed before" relation also provides the programmer with a means of preventing out-of-thin-air executions. If a carried dependency might form an OOTA cycle, the user is responsible for ensuring that the head of the dependency is "placed before" any subsequent use of the carried dependency. Alternatively, if the programmer fails to insert annotations sufficient to prevent a possible OOTA cycle, then the loads in the cycle will be declared to return unspecified values. We fill in the details below.

## § 5.1. Using Annotations to Prevent OOTA

The fact that "placed before" prevents OOTA cycles simply falls out of the existing definition of "happens before", assuming the changes proposed in §4 Fine-Grained Thread-Local Ordering are made. For example, the following new note, derived from Section 32.4 paragraph 9, might be added:

> [ Note: For example, with x and y initially zero,
>
> ```
> // Thread 1:
> r1 = y.load(memory_order::~~relaxed~~ acquire);
> x.store(r1, memory_order::relaxed);
> // Thread 2:
> r2 = x.load(memory_order::~~relaxed~~ acquire);
> y.store(r2, memory_order::relaxed);
> ```
>
> ~~should~~ will not produce r1 == r2 == 42, since the store of 42 to y is ~~only possible if the store to x stores 42, which circularly depends on the store to y storing 42.~~ inter-thread-placed before the store of 42 to x, which is turn inter-thread-placed before the store of 42 to y, and that would form a forbidden "happens before" cycle. —end note ]

## § 5.2. Cycles of Unenforced Dependencies Result in Loads Returning Unspecified Values

If A carries a dependency to B, but A is *not* placed before B, then orderings derived the dependency that is supposedly being carried may not in fact be enforced by the implementation. To describe this, we introduce a notion of "unenforced dependency" as follows:

> An evaluation A *carries-an-unenforced-dependency* to an evaluation B if A carries a dependency to B, and A is not placed before B

As discussed in, e.g., [p0190r3] and [p0462r1], the precise definition of "carries a dependency to" is still being debated. To the best of our knowledge so far, any variant of "carries a dependency to" that properly captures the set of behaviors prone to OOTA will suffice for our proposal. For the moment, we simply stick with "carries a dependency to" as it is currently defined.

We then build a variant of "dependency-ordered before" specifically for unenforced dependencies. We define it by again duplicating the definition of "dependency-ordered before" from Section 6.8.2.1 paragraph 9 and then replacing "carries a dependency" with "carries an unenforced dependency".

> An evaluation A is unenforced-dependency-ordered before an evaluation B if
>
> - A performs a release operation on an atomic object M, and, in another thread, B ~~performs a consume operation on M and~~ reads a value written by any side effect in the release sequence headed by A, or
> - for some evaluation X, A is unenforced-dependency-ordered before X and X carries ~~a~~ an unenforced dependency to B.

A cycle of "unenforced-dependency-ordered-before" represents an execution with possible out-of-thin-air behavior. For this reason, and because "unenforced-dependency-ordered-before" is not used elsewhere, we also define it to be transitively closed.

> - for some evaluation X, A is unenforced-dependency-ordered before X and X is unenforced-dependency-ordered before B.
>
> [ Note: The relation "is unenforced-dependency-ordered before" is analogous to "synchronizes with", but ~~uses release/consume~~ release paired with unenforced-dependency-ordered-before in place of release/acquire. —end note ]

Because the possiblity of out-of-thin-air behavior makes the load return values unpredictable, an execution with an "unenforced-dependency-ordered-before" cycle is declared to result in loads returning unspecified values. We acheive this by modifying Section 32.4 paragraph 9:

> In an execution where a load A is unenforced-dependency-ordered-before itself, the value returned by the load is unspecified. ~~Implementations should ensure~~ In such scenarios, the implementation may not ensure that no "out-of-thin-air" values are computed that circularly depend on their own computation. [ Note: For example, with x and y initially zero,
>
> ```
> // Thread 1:
> r1 = y.load(memory_order::relaxed);
> x.store(r1, memory_order::relaxed);
> // Thread 2:
> r2 = x.load(memory_order::relaxed);
> y.store(r2, memory_order::relaxed);
> ```
>
> ~~should not~~ may produce r1 == r2 == 42, since the store of 42 to y is ~~only~~ possible if the store to x stores 42, which circularly depends on the store to y storing 42, and the implementation cannot in general guarantee that such unexpected behavior will not occur. ~~Note that without this restriction, such an execution is possible.~~ This outcome can be prevented by using stronger synchronization that ensures that the load in each thread is placed before the store in the same thread. —end note ]

## § 5.3. The "OOTA-Free-for-Placed-Before" Theorem

Determining whether a program is prone to OOTA is likely still to be somewhere between difficult and impossible, in much the same way that statically determining the presence or absence of data races and implementing `memory_order_consume` properly are both somewhere between difficult and impossible. However, there is a simple and straightforward recipe that can be followed in all but the most carefully optimized cases:

> A program that ensures every atomic load is "placed before" every atomic store that it is also sequenced before will not admit any executions with out-of-thin-air behavior.

The proof is straightforward: if there are no situations where a load carries-an-unenforced-dependency to a later store, then the OOTA scenario now declared to cause loads to return unspecified values will never occur.

The critierion of the theorem above can be satisfied in any number of ways, but the most straightforward solution is simply to always use `memory_order_load_store` or stronger. This also corresponds to reasoning described in numerous previous OOTA discussions that had proposed, e.g., to globally enforce atomic-load-to-atomic-store order. Another solution for programmers to ensure (manually, with the burden on them to get it right) that the value returned by a

`memory_order_relaxed` load never escapes the local scope. This may be used by expert programmers writing carefully hand-optimized data structures, such as those surveyed in [RAts] or elsewhere.

## § 6. Consequences for Existing Code

Programs that pair `memory_order_release` with forms of "placed before" other than `memory_order_consume` and `memory_order_acquire` will become more constrained, and certain outcomes that are currently permitted would now be forbidden. This is true even though not all of these examples are prone to OOTA behavior. For one example, see the load buffering example with `memory_order_release` stores in §7.1 Load Buffering. We expect that by construction of "placed before", this strengthening is sound with respect to current hardware.

This proposal also declares some unknown number of existing programs with well-defined semantics to suddenly contain loads that return unspecified values. Programs with "unenforced-dependency-ordered before" cycles are well-defined and OOTA-free under the existing specification, but only because the specification makes a promise that implementations cannot currently keep. As such, in spite of the current specification, these programs are already prone to OOTA behavior in practice. The changes described in this proposal simply update the specification to better reflect this reality. Nevertheless, pragmatically speaking, most such programs will continue to behave just as well under our proposal as they currently do under the existing specification.

## § 7. Examples

### § 7.1. Load Buffering

```
// Thread 1:
r1 = y.load(memory_order::relaxed);
x.store(r1, memory_order::relaxed);
// Thread 2:
r2 = x.load(memory_order::relaxed);
y.store(r2, memory_order::relaxed);
```

This program admits an execution in which each load is "unenforced-dependency-ordered before" itself. Therefore, the loads are considered to return unspecified values, possibly including 42. r1 == r2 == 42 or any other possible outcome is therefore possible. In the vast majority of cases, the implementation will actually continue to "do the right thing" here, since OOTA is generally accepted

as unlikely to occur in practice. However, if an implementation were to produce r1 == r2 == 42 in this case, e.g., due to some aggressive compiler optimization, it would now be within its rights to do so. The burden is placed on the programmer to avoid such a scenario.

```
// Thread 1:
r1 = y.load(memory_order::relaxed acquire);
x.store(r1, memory_order::relaxed);
// Thread 2:
r2 = x.load(memory_order::relaxed acquire);
y.store(r2, memory_order::relaxed);
```

The acquire annotations here break the "unenforced-dependency-ordered before" cycle. Therefore, this program has well-defined behavior, and OOTA will be prevented. Notably, the acquire operations have semantic meaning here even though they are not paired with release operations.

```
// Thread 1:
r1 = y.load(memory_order::relaxed load_store);
x.store(r1, memory_order::relaxed);
// Thread 2:
r2 = x.load(memory_order::relaxed load_store);
y.store(r2, memory_order::relaxed);
```

This example, which uses `memory_order_load_store` rather than `memory_order_acquire`, provides the same OOTA-free guarantee but using a cheaper and faster mechanism.

```
// Thread 1:
r1 = y.load(memory_order::relaxed);
x.store(r1, memory_order::relaxed release);
// Thread 2:
r2 = x.load(memory_order::relaxed);
y.store(r2, memory_order::relaxed release);
```

For similar reasoning, this program has well-defined behavior and will not produce OOTA behavior. The release operations have semantic meaning even though they are not paired with acquire operations.

## § 7.2. Histogram

```
void histogram(std::atomic buckets[B], int data[N]) {
    for (int i = 0; i = N; i++) {
        int b = bucket(data[i]);
        buckets[b].fetch_add(1, memory_order_relaxed);
    }
}
```

In this example, the use of `memory_order_relaxed` is safe, because the return values are not used and hence cannot propagate further as unenforced dependencies.

## § 7.3. "Reads-from-untaken-branch"

```
std::atomic x(0), y(0);

// Thread 1:
y.store(x.load(memory_order::relaxed), memory_order::relaxed);

// Thread 2:
bool rfub_occurred;
int r = y.load(memory_order::relaxed);
if (r == 42) {
  rfub_occurred = true;
} else {
  rfub_occurred = false;
  r = 42;
}
x.store(r, memory_order::relaxed);
return rfub_occurred;
```

This example of "reads-from-untaken-branch" (RFUB), a variant of OOTA, is due to Hans Boehm. In a nutshell, the compiler might detect that `r` is always 42 in thread 2, constant-propagate the value 42 to the store to x, and then (becuase there is no annotation or carried dependency preventing it) reorder the store to x before the load of y. This scenario admits the execution in which `rfub_occurred == true`. However, the same reasoning would fail if the `r = 42;` statement were removed from the *untaken* branch, because the compiler would no longer be able to perform the constant propagation. The fact that statements in an untaken branch might influence the set of legal executions is considered unexpected at best.

Under our proposal, the loads in the RFUB example above would be considered to return unspecified values due to the "unenforced-dependency-ordered before" cycle. In spite of the apparent syntactic dependency from the load in each thread to the store in the same thread, neither dependency is *enforced*, and hence neither provides any memory ordering guarantee. Because there is an "unenforced-dependency-ordered before" cycle, the two loads are considered to return unspecified values, and hence the Thread 2 load may well return the value `42` that causes `rfub_occurred == true`.

```
std::atomic x(0), y(0);

// Thread 1:
y.store(x.load(memory_order::relaxed loadstore), memory_order::relaxed);

// Thread 2:
bool rfub_occurred;
int r = y.load(memory_order::relaxed);
if (r == 42) {
  rfub_occurred = true;
} else {
  rfub_occurred = false;
  r = 42;
}
x.store(r, memory_order::relaxed);
return rfub_occurred;
```

This variant of the previous example uses a `memory_order_loadstore` annotation to ensure that thread 1 maintains proper dependency ordering. In this scenario, there is no longer a "unenforced-dependency-ordered before" cycle, and hence the loads return well-specified values. Nevertheless, it remains possible for both loads to return the value 42: the store in Thread 2 may still be reordered before the load, as the Thread 2 dependency is still not enforced.

To analyze this example further, we propose the following interpretation:

- Old intuition: the implementation reordered the memory operations in Thread 2 because of the `r = 42;` statement in the untaken branch
- New intuition: the implementation reordered the memory operations in Thread 2 because the dependency was unenforced (i.e., because the two memory operations were not related by "placed before"), and the implementation was therefore within its rights to do so

In other words, the implementation is free (from a memory model perspective) to reorder the Thread 2 load after the Thread 2 store regardless of whether the `r = 42;` statement is present. The fact that it is

unlikely to do so if the statement is removed becomes an implementation detail irrelevant to the actual formal analysis. Under this interpretation, the example might even be more aptly named "reads from unenforced dependency".

Even with the interpretation above, this RFUB example is likely to remain controversial, as the outcome `rfub_occurred == true` is permitted even though it would never occur under a sequentially consistent execution. We look forward to further discussion on this example.

```
std::atomic x(0), y(0);

// Thread 1:
y.store(x.load(memory_order::relaxed loadstore), memory_order::relaxed);

// Thread 2:
bool rfub_occurred;
int r = y.load(memory_order::relaxed loadstore);
if (r == 42) {
  rfub_occurred = true;
} else {
  rfub_occurred = false;
  r = 42;
}
x.store(r, memory_order::relaxed);
return rfub_occurred;
```

This correction of the RFUB example uses `memory_order_loadstore` annotations to ensure that the load-store reordering in question will not occur, and hence that neither OOTA nor RFUB behavior will be introduced. The outcome `rfub_occurred == true` will not occur in this scenario.

## § 8. Future work

- To the best of our knowledge, our proposed changes are compatible with changes suggested by other proposals, including [p0668r4] and [p0735r0].
- Is "carries a dependency to", or some proposed variant thereof, actually the right notion to use as the basis for "carries an unenforced dependency"?
- Should "unenforced-dependency-ordered before" cycles result in full-blown undefined behavior? Or should they simply result in the loads in such a cycle returning unspecified values as we currently propose?

- What further refinement, if any, is needed to account for the counterintuitive RFUB examples?
- Refine any other wording and/or technical details as necessary, and fix anything I might have messed up

## § 9. Acknowledgements

Hans Boehm, Brian Demsky, Olivier Giroux, Paul McKenney provided valuable discussion. This proposal more broadly also builds off of a ton of previous work by many people on dependency ordering, out-of-thin-air, and C++ memory model work in general.

## § References

### § Informative References

**[GHOSTS]**
Hans-J. Boehm; Brian Demsky. Outlawing Ghosts: Avoiding Out-of-Thin-Air Results. June 13, 2014. URL: https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/42967.pdf

**[N3710]**
H. Boehm, et al.. Specifying the absence of "out of thin air" results (LWG2265). URL: https://wg21.link/n3710

**[N4750]**
Richard Smith. Working Draft, Standard for Programming Language C++. 7 May 2018. URL: https://wg21.link/n4750

**[P0098R1]**
Paul E. McKenney, Torvald Riegel, Jeff Preshing, Hans Boehm, Clark Nelson, Olivier Giroux, Lawrence Crowl. Towards Implementation and Use of memory order consume. 4 January 2016. URL: https://wg21.link/p0098r1

**[P0190R3]**
Paul E. McKenney, Michael Wong, Hans Boehm, Jens Maurer, Jeffrey Yasskin, JF Bastien. Proposal for New memory order consume Definition. 5 February 2017. URL: https://wg21.link/p0190r3

**[P0462R1]**
Paul E. McKenney, Torvald Riegel, Jeff Preshing, Hans Boehm, Clark Nelson, Olivier Giroux, Lawrence Crowl, JF Bastien, Micheal Wong. Marking memory order consume Dependency Chains. 5 February 2017. URL: https://wg21.link/p0462r1

**[P0668R4]**

Hans-J. Boehm, Olivier Giroux, Viktor Vafeiades. Revising the C++ memory model. 24 June 2018. URL: https://wg21.link/p0668r4

**[P0735R0]**

Will Deacon. Interaction of memory_order_consume with release sequences. 2 October 2017. URL: https://wg21.link/p0735r0

**[P0750R1]**

JF Bastien, Paul E. McKenney. Consume. 11 February 2018. URL: https://wg21.link/p0750r1

**[RAts]**

Matthew D. Sinclair; Johnathan Alsop; Sarita V. Adve. Chasing Away RAts: Semantics and Evaluation for Relaxed Atomics on Heterogeneous Systems. June 24, 2018. URL: http://rsim.cs.illinois.edu/Pubs/17-ISCA-RAts.pdf

**[WebKit]**

JF Bastien; Filip Jerzy Pizło. WebKit source: WTF Atomics.h. June 2, 2017. URL: https://trac.webkit.org/browser/webkit/trunk/Source/WTF/wtf/Atomics.h?rev=+217722#L342